

Fundacao BullMQ no XUA Delivery

Resumo

Nesta etapa, a fila foi preparada, mas os fluxos reais do sistema ainda nao foram migrados para ela.

Em termos simples:

- antes, o projeto nao tinha infraestrutura de fila pronta para uso
- agora, o projeto ja tem BullMQ instalado, fila criada e worker separado funcionando
- os processos atuais ainda continuam rodando do jeito antigo
- a proxima etapa sera conectar processos reais nessa fila, de forma gradual e segura

Ou seja: a fundacao foi criada. Agora existe o trilho. O proximo passo, quando fizer sentido, sera colocar os processos atuais para rodarem por esse trilho.

O que foi implementado

1. BullMQ foi adicionado ao projeto

Foi instalada a dependencia BullMQ no pacote da API.

Isso permite que o sistema:

- publique jobs em uma fila Redis
- tenha workers consumindo esses jobs
- controle retry, backoff e ciclo de vida de jobs

Arquivo principal:

- apps/api/package.json

2. Foi criada a camada de fila

Foi criada uma base central para a infraestrutura de fila em `apps/api/src/infra/queue`.

Essa camada centraliza:

- conexao da fila
- nomes das filas
- contratos dos jobs
- criacao das filas
- producer para jobs internos

Arquivos principais:

- apps/api/src/infra/queue/config.ts
- apps/api/src/infra/queue/connection.ts
- apps/api/src/infra/queue/contracts.ts
- apps/api/src/infra/queue/queues.ts

- apps/api/src/infra/queue/internal-jobs.producer.ts
- apps/api/src/infra/queue/index.ts

3. Foi criado um worker separado da API web

Foi criado um processo worker proprio em `apps/api/src/worker/index.ts`.

Esse worker:

- sobe separado do servidor HTTP
- escuta a fila de jobs internos
- processa jobs em segundo plano
- faz shutdown limpo
- registra logs de inicio, sucesso e falha

Arquivos principais:

- apps/api/src/worker/index.ts
- apps/api/src/worker/processors/internal-jobs.processor.ts

4. Os handlers antigos ja podem ser reaproveitados pelo worker

O processor criado nao inventa uma regra de negocio nova. Ele reaproveita os jobs que ja existiam no sistema.

Hoje ele ja sabe chamar:

- otp-cleanup
- subscription-generation
- subscription-expiry

Esses handlers continuam existindo e funcionando como antes.

Arquivos chamados pelo worker:

- apps/api/src/jobs/otp-cleanup-job.ts
- apps/api/src/jobs/subscription-job.ts
- apps/api/src/jobs/subscription-expiry-job.ts

5. Foi criado um job noop apenas para teste

Foi adicionado um job interno chamado `noop`.

Ele serve apenas para validar o caminho completo da fila sem tocar em banco, pagamento ou regras de negocio sensiveis.

Esse job:

- entra na fila
- e consumido pelo worker
- termina com sucesso
- gera logs de inicio e conclusao

Scripts adicionados e o que cada um faz

Foram adicionados dois scripts novos no pacote da API:

- `dev:worker`
- `worker`

Eles não substituem os scripts antigos da API web. Eles existem para subir o processo de worker separadamente.

Script `dev:worker`

Comando:

```
npm run dev:worker -w @xua/api
```

Esse script é voltado para desenvolvimento.

Ele:

- carrega o arquivo `.env`
- usa `tsx` em modo watch
- reinicia automaticamente quando arquivos do worker mudam
- executa `src/worker/index.ts`

Uso ideal:

- desenvolvimento local
- teste manual da fila
- ajuste de processors e contracts

Script `worker`

Comando:

```
npm run worker -w @xua/api
```

Esse script é voltado para execução contínua, sem watch.

Ele:

- executa `src/worker/index.ts`
- sobe o processo worker uma vez
- fica aguardando jobs na fila
- usa o shutdown limpo definido no código

Uso ideal:

- produção

- staging
- servico separado no Render

Diferenca entre `dev:worker` e `worker`

Resumo simples:

- `dev:worker` e para desenvolvimento local com hot reload
- `worker` e para execucao continua em ambiente de deploy

E o mesmo processo logico, mas em modos diferentes de execucao.

Scripts antigos continuam iguais

Os scripts antigos continuam com o papel original:

- `dev`: sobe a API web em modo watch
- `start`: sobe a API web em modo normal

Resumo completo:

- `dev` -> API web em desenvolvimento
- `start` -> API web em execucao normal
- `dev:worker` -> worker em desenvolvimento
- `worker` -> worker em execucao normal

Como isso deve ser deployado no Render

O modelo correto no Render e subir processos separados.

Estrutura recomendada

Hoje o desenho correto e este:

1. um servico web para a API
2. um servico worker separado para o BullMQ
3. os cron jobs atuais permanecem temporariamente como estao

Ou seja: nao e uma unica instancia fazendo os dois papeis.

O recomendado e:

- **API web**: roda `apps/api/src/server/index.ts`
- **worker**: roda `apps/api/src/worker/index.ts`

Por que separar em duas instancias

Porque sao responsabilidades diferentes.

A API web:

- atende HTTP
- responde requests

- expõe endpoints
- precisa manter latencia previsivel

O worker:

- nao atende HTTP
- fica consumindo jobs continuamente
- pode ter comportamento de carga e concorrencia diferente
- pode crescer separadamente no futuro

Se colocar tudo no mesmo processo, voce perde a separacao que justamente estamos criando.

Como isso ficaria no Render

Hoje o `render.yaml` tem apenas:

- um servico web chamado `xua-api`
- cron jobs chamando endpoints HTTP internos

Para usar o worker em producao, o proximo passo sera adicionar um novo servico, por exemplo:

- type: worker
- name: xua-api-worker
- buildCommand: `npm install && npx prisma generate`
- startCommand: `npm run worker -w @xua/api`

Esse servico worker usaria as mesmas variaveis essenciais:

- `DATABASE_URL`
- `REDIS_URL` ou `QUEUE_REDIS_URL`
- `NODE_ENV`
- quaisquer secrets necessarios aos jobs que ele realmente for executar

O que acontece se eu apenas committar isso e fizer deploy agora

Essa e a parte mais importante operacionalmente.

Se voce apenas fizer commit e deploy com o `render.yaml` atual

O que vai acontecer:

- a API web vai subir normalmente
- o BullMQ sera instalado como dependencia
- os arquivos novos vao existir no codigo deployado
- o worker **nao** vai subir automaticamente
- nenhum fluxo atual vai passar a usar fila sozinho

Ou seja:

- o sistema continua funcionando como antes
- o servico web atual continua sendo iniciado por `startCommand: npx tsx apps/api/src/server/index.ts`

- como o servidor web nao inicia `src/worker/index.ts`, o worker fica inativo

Em resumo:

somente committar e deployar nao ativa o worker em producao.

Isso quebra o sistema atual?

Nao.

Como os endpoints atuais ainda nao foram entrelacados com a fila, o comportamento esperado e:

- tudo continua como antes
- sem uso real de BullMQ no fluxo principal
- sem regressao funcional causada apenas por essa fundacao

Se eu criar o servico worker no Render agora, mesmo sem integrar os processos reais

Nesse caso:

- o worker vai subir
- ele vai se conectar ao Redis
- ele vai ficar aguardando jobs
- mas ele ficara ocioso na maior parte do tempo, porque os fluxos reais ainda nao publicam jobs automaticamente

Ou seja:

- **web sem worker:** sistema continua como antes
- **web com worker separado:** sistema continua como antes, mas agora com o consumidor pronto e esperando jobs

O que ainda falta para o deploy ficar operacional de verdade

Para dizer que o worker esta realmente participando da aplicacao em producao, ainda faltam tres passos:

1. conectar pelo menos um processo real de baixo risco para publicar jobs
2. adicionar o servico worker no Render
3. validar producer -> Redis -> worker em ambiente de deploy

Enquanto isso nao for feito, a fundacao existe, mas o uso real ainda nao.

O que muda hoje no sistema

Quase nada muda para o usuario final nesta etapa.

O que entrou agora e infraestrutura, nao mudanca de produto.

Na pratica:

- a API continua funcionando do mesmo jeito
- os endpoints internos de jobs continuam rodando do jeito antigo
- os cron jobs atuais continuam como estao

- pagamentos continuam intocados
- webhooks continuam intocados

O que mudou foi a capacidade tecnica do sistema:

- agora o projeto ja consegue enfileirar jobs
- agora o projeto ja consegue processar jobs em um processo separado da API web
- agora ja existe a base para migrar fluxos reais aos poucos, sem reescrever o sistema todo

O que ainda nao foi feito

Esta parte e importante para alinhar expectativa.

Ainda nao foi feito:

- ligar endpoints HTTP reais na fila
- trocar os cron jobs atuais para usarem BullMQ
- mover pagamentos para fila
- mover webhook para fila
- criar dashboard operacional de fila
- escalar multiplas instancias com adapter Redis no Socket.IO

Entao o sistema ainda nao esta usando BullMQ no fluxo principal. Ele apenas esta pronto para comecar a usar.

Por que essa abordagem e a correta

Essa ordem reduz risco.

Se a fila fosse plugada direto em pagamento ou webhook logo de inicio, qualquer erro de contrato, processamento ou retry poderia afetar uma parte muito sensivel do sistema.

Ao criar a fundacao primeiro, o projeto ganha:

- separacao entre processo web e processo worker
- estrutura de fila organizada
- contratos de jobs
- logs e comportamento previsivel
- possibilidade de testar tudo antes de tocar no fluxo financeiro

Em resumo: primeiro montamos a infra. Depois conectamos processos reais nela, um por vez.

Como ver isso funcionando na pratica

O teste mais seguro e com o job `noop`.

1. Subir o worker

Comando:

```
npm run dev:worker -w @xua/api
```

O esperado e ver algo como:

```
XUA worker started
queue: internal-jobs
prefix: xua:development:queue
concurrency: 1
```

2. Enfileirar um job de teste

Comando:

```
node --env-file=apps/api/.env --import tsx -e "const producerMod = await
import('./apps/api/src/infra/queue/internal-jobs/producer.ts'); const {
enqueueInternalJob } = producerMod.default; const result = await
enqueueInternalJob({ jobName: 'noop', source: 'ops' });
console.log(JSON.stringify(result)); process.exit(0);"
```

O esperado e receber algo assim:

```
{"id":"1","name":"noop","correlationId":"..."}
```

3. Confirmar o processamento no worker

No terminal do worker, o esperado e ver:

```
Internal job started
jobName: noop

Internal job completed
jobName: noop
```

O que esse teste prova

Esse teste prova tres coisas importantes:

1. o producer conseguiu publicar um job na fila
2. o Redis armazenou e coordenou esse job
3. o worker separado consumiu e concluiu o job

Isso mostra que o caminho completo ja existe:

producer -> fila Redis -> worker -> conclusao

O que o time deve entender neste momento

O estado atual do projeto e este:

- a fundacao BullMQ ja existe
- o worker separado ja existe
- o caminho de fila ja funciona
- os fluxos reais ainda nao foram migrados

Traduzindo em uma frase:

"Criamos a fila. Agora falta usar a fila."

Mas isso sera feito depois, por etapas, sem quebrar o sistema atual.

Proximo passo natural

O proximo passo mais seguro e conectar um processo real de baixo risco nessa fila, mantendo o fallback antigo por algum tempo.

O melhor candidato inicial e:

- otp-cleanup

Porque:

- nao e fluxo financeiro
- ja existe como job separado
- e facil validar
- o rollback e simples

Depois disso, viriam os demais jobs internos. So depois faria sentido pensar em webhook e pagamento.